JavaScript Essentials & Advanced — Course Summary

> Based on javascript.pit-graduaten.be
> Enriched with course slides from Thomas More (PIT Graduaten).
> C# comparisons included where relevant.

🟡 Essentials

Lecture 1 — Syntax & Control Structures

Background

JavaScript is standardized by Ecma International under the specification **ECMA-262** — currently at ECMAScript 2024 (15th edition). Not every feature in the latest spec is supported in all browsers. Always check MDN for compatibility.

JavaScript is one of the three languages every web developer must know: **HTML** defines content, **CSS** defines layout, and **JavaScript** defines behaviour. Common use cases include DOM manipulation, form validation, dynamic menus, Bootstrap interactivity (modals, carousels), Ajax web apps, and full Single Page Applications.

Running a script

In the first three lectures, scripts run locally using **Bun** (not the browser). From lecture 4 onwards, JS runs client-side in the browser. From lecture 6 (Advanced), Node.js is introduced as the runtime.

To run a script with Bun in VS Code:

1. Open the folder containing the script in VS Code
2. Open the integrated terminal (`Terminal > New Terminal`)
3. Type `bun hello_world.js` and press Enter

> **Tip:** Tab autocomplete works — `bun he` + Tab expands to `bun hello_world.js`. Arrow keys navigate command history.

Variables

Variables use `const` (immutable reference, preferred) and `let` (mutable). There's no `int`, `string`, `bool` etc. — JavaScript is dynamically typed.

`var` **must never be used in modern JS.** It has global scope and can be both reassigned and redeclared anywhere, which leads to unpredictable bugs. *(See Addendum A1 for the full scope table.)*

```
const admin = "Niels";      // cannot be reassigned
let score = 0;              // will be updated later
score = 10;                 // ✅ fine

// var — NEVER use
var x = 5;
var x = 10;                 // redeclaring is valid with var — this is the danger
```

Always prefer `const`. Use `let` only when you know the value will change.

Types

Types are: `number` (covers both int and float), `string`, `boolean`, `object`, `undefined`, and `null`. Use `typeof` to inspect the type of a variable at runtime:

```
typeof 42          // "number"
typeof "hello"     // "string"
typeof true        // "boolean"
typeof undefined   // "undefined"
typeof null        // "object"  ⚠️ historical bug — null is NOT an object
typeof {}          // "object"
```

The distinction between `null` (explicitly no value) and `undefined` (declared but not assigned, or doesn't exist) is important and unique to JS. *(See Addendum A2.)*

Strings

Strings can use single or double quotes interchangeably. Use a backslash `\` as an escape character when you need the same quote type inside the string:

```
const escaped = 'I\'ve got no right...';   // escaped single quote
```

Concatenate strings with `+`, or use **template literals** (backticks) — equivalent to C#'s `$"..."`:

```
const one = 'Hello, ';
const two = 'how are you?';
const joined = one + 'I am fine — ' + two;         // concatenation
const template = `${one} I am fine — ${two}`;      // template literal (preferred)
```

Operators

Category	Operators
Arithmetic	`* / + - % ++ -- ** += -=`
Comparison	`== === < > <= >= != !==`
Logical	`!` (not), `&&` (and), `||` (or)
Advanced	`??` (nullish coalescing), `?.` (optional chaining), `? :` (ternary)

Always use `===` (strict equality, checks type) over `==` (loose equality). Same for `!==` over `!=`.

Console output

`console.log()` accepts multiple arguments:

```
console.log("string")
console.log("string", variable, 42)    // prints all, space-separated
console.warn("warning message")
console.error("error message")
```

User input: `prompt()` and `confirm()`

```
// prompt() always returns a STRING — convert when you need a number
const name = prompt('What is your name?', 'John Doe');  // second arg = default
const age = Number(prompt('How old are you?'));          // convert to number!

// confirm() returns a BOOLEAN
const ok = confirm('Are you sure?');    // true = OK, false = Cancel
```

> In Bun (terminal), these show as console prompts. In the browser, they open popup dialogs. *(See Addendum A3 for the common bug.)*

```
// Exercise: prompt + confirm combined — building a simple name-confirmation loop
const voornaam = prompt('Geef je voornaam in: ')
const achternaam = prompt('Geef je achternaam in: ')

console.log(`Je naam is ${voornaam} ${achternaam}`)

const isCorrect = confirm('Is dit correct?')

if (isCorrect) {
  console.log('Bedankt voor de bevestiging!')
} else {
  console.log('Sorry voor de verwarring, voer het programma opnieuw uit om je correcte naam in t
}


// Exercise: Number() conversion + all basic arithmetic operators
const getal1 = Number(prompt('Geef een eerste getal in: ', '0'))
const getal2 = Number(prompt('Geef een tweede getal in: ', '1'))

console.log(`De som van ${getal1} en ${getal2} is ${getal1 + getal2}`)
console.log(`Het verschil tussen ${getal1} en ${getal2} is ${getal1 - getal2}`)
console.log(`Het product van ${getal1} en ${getal2} is ${getal1 * getal2}`)
console.log(`Het quotiënt van ${getal1} en ${getal2} is ${getal1 / getal2}`)
console.log(`De rest van de deling van ${getal1} door ${getal2} is ${getal1 % getal2}`)
```

Conditionals

```
if (time < 10) {
  greeting = "Good morning";
} else if (time < 20) {
  greeting = "Good day";
} else {
  greeting = "Good evening";
}

// Ternary — short form
const result = (score === 80) ? 'Correct!' : 'Try again';
```

```
// Exercise: modulo to check even/odd — shows ternary vs if/let pattern
const getal = Number(prompt('Geef een getal in: '))

// Cleaner alternative using ternary:
// const isEven = getal % 2 === 0 ? 'even' : 'oneven'
let isEven = 'oneven'
if (getal % 2 === 0) {
  isEven = 'even'
}
console.log(`${getal} is een ${isEven} getal`)


// Exercise: if / else if / else with ** (exponentiation operator) and Math.pow
const getal = Number(prompt('Geef een getal in tussen 10 en 20: '))

if (getal < 10) {
    console.log('Dit getal is kleiner dan 10')
} else if (getal > 20) {
    console.log('Dit getal is groter dan 20')
} else {
  console.log(`Het kwadraat van dit getal is ${getal ** 2}`)
  // Equivalent: Math.pow(getal, 2)
}


// Exercise: Math.min / Math.max / derived middle value
const getal1 = Number(prompt('Geef een eerste getal in: ', '0'))
const getal2 = Number(prompt('Geef een tweede getal in: ', '0'))
const getal3 = Number(prompt('Geef een derde getal in: ', '0'))

const kleinsteGetal = Math.min(getal1, getal2, getal3)
const grootsteGetal = Math.max(getal1, getal2, getal3)
const middelsteGetal = getal1 + getal2 + getal3 - kleinsteGetal - grootsteGetal

console.log(`Het kleinste getal is ${kleinsteGetal}`)
console.log(`Het middelste getal is ${middelsteGetal}`)
console.log(`Het grootste getal is ${grootsteGetal}`)
```

When to use `switch` vs `if`:

Use `if` when...	Use `switch` when...
Testing ranges or complex expressions	Comparing one variable to fixed values
Multiple different conditions	Several cases share the same logic

```
switch (day) {
  case 'Monday':
    console.log('Start of week');
    break;
  case 'Friday':
    console.log('Almost weekend');
    break;
  default:
    console.log('Midweek');
}
```

> ⚠️ Always include `break` — without it, execution falls through to subsequent cases!

```
// Exercise: switch(true) for range-based conditions (alternative to if/else if)
const getal = Number(prompt('Geef een getal in tussen 10 en 20: '))

switch (true) {
  case getal < 10:
    console.log('Dit getal is kleiner dan 10')
    break
  case getal > 20:
    console.log('Dit getal is groter dan 20')
    break
  default:
    console.log(`Het kwadraat van dit getal is ${getal ** 2}`)
    break
}
```

```
// Exercise: switch(true) with multiple ranges — ticket pricing tiers
const leeftijd = Number(prompt('Geef je leeftijd in: '))

switch (true) {
  case leeftijd < 5:
    console.log("Je ticket is gratis.")
    break
  case leeftijd < 12:
    console.log("Je betaalt je ticket aan halve prijs.")
    break
  case leeftijd <= 55:
    console.log("Je krijgt geen korting.")
    break
  case leeftijd > 55:
    console.log("Je ticket is gratis.")
    break
  default:
    console.log("Je hebt geen geldige leeftijd ingegeven.")
    break
}
```

```
// Exercise: switch on string value — planet weight calculator
const mass = Number(prompt('Geef je massa in: '))
console.log('Op welke planeet wil je jouw gewicht berekenen?')
console.log('    A. Maan  B. Jupiter  C. Mars  D. Venus  E. Neptunus')
const planet = prompt('Jouw keuze: ')

let weight = 0
let name = ''
switch (planet) {
  case "A": weight = mass * 1.625;  name = 'Maan';     break
  case "B": weight = mass * 25.93;  name = 'Jupiter';  break
  case "C": weight = mass * 3.728;  name = 'Mars';     break
  case "D": weight = mass * 8.872;  name = 'Venus';    break
  case "E": weight = mass * 11.28;  name = 'Neptunus'; break
}

if (weight === 0 || isNaN(weight)) {
  console.log('Je hebt geen geldige planeet gekozen of je opgegeven'+
  'massa kon niet geconverteerd worden naar een getal.')
} else {
  console.log(`Jouw gewicht op ${name} is ${weight} Newton`)
}
```

Loops

```
for (let i = 0; i < 10; i++) { ... }        // bounded — use when count is known
for (let i = 10; i >= 0; i--) { ... }       // negative step also works

while (condition) { ... }                    // unbounded — condition checked before
do { ... } while (condition);                // runs at least once
```

`break` exits the loop immediately. `continue` skips the rest of the current iteration.

> ⚠️ **Infinite loop pitfall with `continue`:** if `continue` appears before the counter increment in a `while` loop, the counter never increments and the loop runs forever.

```
// Exercise: while with confirm — loop until user confirms correct input
let voornaam, achternaam, isCorrect

while (!isCorrect) {
  voornaam = prompt('Geef je voornaam in: ')
  achternaam = prompt('Geef je achternaam in: ')
  console.log(`Je naam is ${voornaam} ${achternaam}`)
  isCorrect = confirm('Is dit correct?')
  if (!isCorrect) console.log('Sorry voor de verwarring, geef je naam opnieuw in.')
}
console.log('Bedankt voor de bevestiging!')


// Exercise: for loop building a string character by character
const getal = Number(prompt('Geef een getal in: '))

let output = ''
for (let i = 0; i < getal; i++) {
  output += '+'
}
console.log(output)


// Exercise: while loop — keep adding 12 until >= 100
let getal = Number(prompt('Geef een getal in: '))
let uitvoer = `${getal} `

while (getal + 12 < 100) {
  getal += 12
  uitvoer += `${getal} `
}
console.log(uitvoer)
```

```
// Exercise: while with Math.floor — lion population growing 15% per year
const showYears = confirm('Wil je het aantal jaren zien?')

let nbLions = 50
let singleRowOutput = `${nbLions} `
let year = 0

while (nbLions < 1000) {
  nbLions = nbLions + Math.floor(nbLions * 0.15)
  singleRowOutput += `${nbLions} `
  year++
  if (showYears) console.log(`${year}: ${nbLions}`)
}
if (!showYears) console.log(singleRowOutput)


// Exercise: nested for + while — Collatz sequence with padEnd formatting
let limit = Number(prompt('Hoeveel rijen moeten geprint worden?'))

for (let n = 2; n <= limit; n++) {
  let current = n
  let row = `${(n + ':').padEnd(5, ' ')}`

  while (current !== 1) {
    row += current.toString().padEnd(5, ' ')
    current = current % 2 === 0 ? current / 2 : 3 * current + 1
  }
  console.log(row)
}
```

Functions

```
// Declaration – hoisted (can be called before it appears in the file)
function greet(name, title = 'Gegradueerde') {
  return `Hello ${name}, ${title}`;
}

// Passing a function as a variable – omit the parentheses
function runTenTimes(fn) {
  for (let i = 0; i < 10; i++) fn();
}
runTenTimes(greet);
runTenTimes(function() { console.log('anonymous'); });  // anonymous function inline
```

Hoisting: function declarations are loaded into memory before code runs, so you can call them before they appear in the file. Arrow functions and function expressions are NOT hoisted. *(See Addendum A4.)*

Variable scope: a variable declared inside a function with `const` / `let` is local — it does not affect outer variables of the same name. However, a function CAN read and modify outer variables that weren't re-declared inside it. *(See Addendum A8.)*

```
// Exercise: functions declared and used before definition (hoisting)
const getal1 = Number(prompt("Geef getal1 in", "0"));
const getal2 = Number(prompt("Geef getal2 in", "0"));

const som = berekenSom(getal1, getal2);
const verschil = berekenVerschil(getal1, getal2);
const product = berekenProduct(getal1, getal2);

const res = `De ingegeven getallen zijn ${getal1} en ${getal2}.
De som is: ${som}
Het verschil is: ${verschil}
Het product is: ${product}`;
console.log(res)

function berekenSom(getal1, getal2)      { return getal1 + getal2; }
function berekenVerschil(getal1, getal2) { return getal1 - getal2; }
function berekenProduct(getal1, getal2)  { return getal1 * getal2; }
```

```
// Exercise: function with a for loop — multiplication table
const getal = Number(prompt("Geef getal", "7"));

function tafel(getal) {
  for (let i = 1; i <= 10; i++) {
    console.log(i + " x " + getal + " = " + i * getal);
  }
}
tafel(getal);


// Exercise: higher-order function — accepts an operation function + optional limit
function generateSequence(number, operation, limit = 100) {
  let sequence = `${number} `;
  while (number < limit) {
    number = operation(number);
    sequence += `${number} `;
  }
  return sequence
}

console.log('Maal 2 (met default):')
const timesTwo = generateSequence(1, function(number) {
  return number * 2;
})
console.log(timesTwo)

console.log('Kwadraat (met de grens op 1000):')
const squared = generateSequence(
  2,
  function(number) { return number ** 2; },
  1000
)
console.log(squared)
```

```
// Exercise: higher-order function — callback receives both value AND iteration index
function generateSequence(number, operation, limit = 100) {
  let i = 1;
  let sequence = `${number} `;
  while (number < limit) {
    number = operation(number, i);
    sequence += `${number} `;
    i++;
  }
  return sequence
}

console.log('Afwisselend plus 5 en min 2 (grens op 30):')
const plus5Minus2 = generateSequence(
  2,
  function(number, i) {
    return i % 2 === 1 ? number + 5 : number - 2;
  },
  30
)
console.log(plus5Minus2)


// Exercise: simple recursion — Fibonacci (note: slow without memoization for large n)
function fibonacci(n) {
  if (n === 0) return 0
  if (n === 1) return 1
  return fibonacci(n - 1) + fibonacci(n - 2)
}

const n = Number(prompt('Welk Fibonacci-getal wil je berekenen?'))
console.log(fibonacci(n))
```

```
// Exercise: recursion with multiple base cases — Levenshtein edit distance
function levenshtein(string1, string2) {
  if (string1.length === 0) return string2.length;
  if (string2.length === 0) return string1.length;

  if (string1.charAt(0) === string2.charAt(0)) {
    return levenshtein(string1.substring(1), string2.substring(1));
  }

  const deletion     = levenshtein(string1.substring(1), string2);
  const insertion    = levenshtein(string1, string2.substring(1));
  const substitution = levenshtein(string1.substring(1), string2.substring(1));

  return 1 + Math.min(insertion, deletion, substitution);
}

const string1 = prompt('Voer de eerste string in:');
const string2 = prompt('Voer de tweede string in:');
console.log(`De Levenshtein afstand tussen "${string1}" en "${string2}" is ${levenshtein(string1
```

Lecture 2 — Arrow Functions, Arrays & Sets

Arrow functions

Arrow functions are a compact alternative to traditional function expressions, most useful as callbacks:

```
const bar = () => 'Hello Bar';            // no params — parens required
const double = x => x * 2;                // one param — parens optional
const add = (a, b) => a + b;              // multiple params — parens required
const compute = (a, b) => {               // multi-line — braces + return required
  const result = a + b;
  return result;
};
```

Arrow functions **do not have their own `this`** — they inherit it from the surrounding scope. This is why they are always preferred for callbacks (see Lecture 3).

Higher-Order Functions

A **higher-order function** accepts another function as an argument or returns one. `map` , `filter` , and `reduce` are all higher-order functions — they replace explicit loops for data transformation:

```
higherOrderFunction(() => {
  // arrow function passed as callback — most common in modern JS
});
```

Arrays

Arrays are ordered, index-based collections. Index starts at 0. JS arrays are dynamic — no fixed size (like C#'s `List<T>`).

```
const empty = [];
const tens = [10, 20, 30, 40, 50];
const sparse = [1, , 3];              // empty slot at index 1
```

> ⚠️ **Never mix types in an array** (e.g. `['Hello', 10, true]`). While technically valid, it makes code unpredictable and difficult to type-check in TypeScript.

`length` **property** — can be read or set. Setting it smaller **truncates** the array:

```
const z = ['a', 'b', 'c', 'd'];
z.length = 2;   // z is now ['a', 'b'] — 'c' and 'd' are gone
z.length = 0;   // z is now []
```

Iteration — important differences between loop types:

Loop	Shows empty slots as `undefined` ?	Supports `break` / `continue` ?
`for` / `while` / `do...while`	✅ Yes	✅ Yes
`for...of`	✅ Yes	✅ Yes
`for...in`	❌ Skips them	✅ Yes
`.forEach()`	❌ Skips them	❌ No

Use `for...of` as your default (like C#'s `foreach`). Avoid `for...in` on arrays — it iterates over **string indices** and also picks up prototype properties. *(See Addendum A6.)*

Key array methods:

Method	Description	C# equivalent
`.push(x)`	Add to end	`List.Add()`
`.pop()`	Remove from end	—
`.unshift(x)`	Add to beginning	`List.Insert(0, x)`
`.shift()`	Remove from beginning	—
`.sort()`	Sort in place — ⚠️ alphabetical by default!	`List.Sort()`
`.reverse()`	Reverse in place	`List.Reverse()`
`.join(sep)`	Array → single string	`string.Join()`
`.fill(val, start, end)`	Fill range with value	—
`.filter(predicate)`	New filtered array	LINQ `.Where()`
`.map(callback)`	New transformed array	LINQ `.Select()`
`.reduce(callback, init)`	Accumulate to single value	LINQ `.Aggregate()`
`.find()`	First matching element	LINQ `.FirstOrDefault()`
`.findIndex()`	Index of first match	—
`.slice(start, end)`	New sub-array (non-destructive)	—
`.concat()`	Merge two arrays into a new array	—
`at(-1)`	Last element (negative index supported)	—

> ⚠️ **`sort()` gotcha with numbers:** numbers are compared as strings by default. To sort numerically, pass a comparator. *(See Addendum A5.)*

> ⚠️ **`sort()` and `reverse()` modify the original array in place** — they do not return a new copy.

```
// Exercise: basic array operations – length, indexed access, sort, push, join
const computerScientists = ['Alan Turing', 'Ada Lovelace', 'Charles Babbage', 'Donald Knuth', '(

console.log('a. ', computerScientists.length)

console.log('\nb.')
console.log('  Eerste element:', computerScientists[0])
console.log('  Derde element:', computerScientists[2])
console.log('  Vijfde element:', computerScientists[4])

computerScientists.sort()
console.log('\nc. ', computerScientists)

const extraName = prompt('Geen een extra naam op: ')
computerScientists.push(extraName)
console.log('\nd. ', computerScientists)

console.log('\ne. ', computerScientists.join(';'))


// Exercise: palindrome check using .at(-1) for negative indexing
function isPalindrome(array) {
  for (let i = 0; i < array.length / 2; i++) {
    if (array[i] !== array?.at(-i - 1)) {
      return false
    }
  }
  return true
}

const array = prompt('Geef een array in, gescheiden door komma\'s:').split(', ')
if (isPalindrome(array)) {
  console.log('De array is een palindroom.')
} else {
  console.log('De array is geen palindroom.')
}
```

filter() :

```
const ages = [32, 16, 40, 7, 56];
const adults = ages.filter(age => age >= 18);   // [32, 40, 56]
```

```
// Exercise: filter + map on an array of exam marks
const marks = [23, 45, 5, 39, 48, 59, 76, 49, 57, 89, 60, 82]

console.log('Originele cijfers:', marks)
console.log('Op 20:', marks.map(mark => mark / 5))

const passed = marks.filter(mark => mark >= 50)
console.log(`Er zijn ${passed.length} geslaagde studenten:`, passed)

const failed = marks.filter(mark => mark < 50)
console.log(`Er zijn ${failed.length} gebuisde studenten:`, failed)
```

`map()` — callback also optionally receives the index:

```
const nums = [1, 2, 3, 4, 5];
const squares = nums.map(n => n * n);
const mixed = nums.map((n, index) => index % 2 === 0 ? n * n : n / 2);
```

`reduce()` — the initial value (second argument) matters significantly. *(See Addendum A7.)*

```
const sum = nums.reduce((acc, cur) => acc + cur, 0);    // 15
const product = nums.reduce((acc, cur) => acc * cur, 1); // 120 — initial must be 1!
```

```
// Exercise: building stats from user input — min, max, sum, average using reduce
const numbers = []
let input = prompt('Geef een getal in, of druk op enter om te stoppen:', '')
while (input !== '') {
  const parsedNumber = Number(input)
  if (!Number.isNaN(parsedNumber)) numbers.push(Number(input))
  input = prompt('Geef een getal in, of druk op enter om te stoppen:', '')
}

let min = numbers[0]
let max = numbers[0]
for (const number of numbers) {
  if (number < min) min = number
  if (number > max) max = number
}

const sum = numbers.reduce((acc, number) => acc + number, 0)
const average = sum / numbers.length

console.log('Je hebt volgende getallen ingegeven:', numbers)
console.log('Het kleinste getal is:', min)
console.log('Het grootste getal is:', max)
console.log('De som van de getallen is:', sum)
console.log('Het gemiddelde van de getallen is:', average)


// Exercise: above extended with sort + median calculation
numbers.sort((a, b) => a - b)   // ⚠️ must use comparator for numbers (see A5)

let median = 0
if (numbers.length % 2 === 0) {
  const middle = numbers.length / 2
  median = (numbers[middle - 1] + numbers[middle]) / 2
} else {
  median = numbers[numbers.length / 2 - 0.5]
}
console.log('De mediaan van de getallen is:', median)
```

`join()` and `split()`:

```
['a@x.be', 'b@x.be'].join(';');      // "a@x.be;b@x.be"
"a@x.be;b@x.be".split(';');          // ['a@x.be', 'b@x.be']
```

```
// Exercise: split input into two arrays based on prefix — 'g' for groente, 'f' for fruit
const vegetables = []
const fruit = []

let input = prompt('Geef een fruit of groente in, of druk op enter om te stoppen:', '')
while (input !== '') {
  const split = input.split(' ')
  if (split.length === 2 && split[0] === 'g') vegetables.push(split[1])
  else if (split.length === 2 && split[0] === 'f') fruit.push(split[1])
  input = prompt('Geef een fruit of groente in, of druk op enter om te stoppen:', '')
}

console.log('Groenten:', vegetables)
console.log('Fruit:', fruit)


// Exercise: Caesar cipher (ROT3) — indexOf + modulo for wrap-around
const alphabet = ['a','b','c','d','e','f','g','h',
'i','j','k','l','m','n','o','p','q','r','s','t','u','v','w','x','y','z'];
const input = prompt('Geef een zin in:').split('')
const output = []

for (let i = 0; i < input.length; i++) {
  const letter = input[i]
  if (letter === ' ') { output.push(letter); continue }
  const index = (alphabet.indexOf(letter) + 3) % 26
  output.push(alphabet[index])
}
console.log('Rot3 (Caesarcijfer) encoded tekst:', output.join(''))
```

Sets

Sets store **unique, unordered** values. Checking membership with `.has()` is much faster than scanning an array.

```
const langs = new Set(['JS', 'Python', 'JS']); // duplicate removed → 2 items
langs.add('C#');
langs.delete('Python');
langs.has('JS');     // true
langs.clear();       // empties the set

for (const lang of langs) { console.log(lang); }
langs.forEach(lang => console.log(lang));

// Set math
setA.union(setB)          // all elements from both
setA.intersection(setB)   // only elements in both
setA.difference(setB)     // elements in A but not B


// Exercise: same fruit/vegetable split using Set instead of array — duplicates prevented
const vegetables = new Set()
const fruit = new Set()

let input = prompt('Geef een fruit of groente in, of druk op enter om te stoppen:', '')
while (input !== '') {
  const split = input.split(' ')
  if (split.length === 2 && split[0] === 'g') vegetables.add(split[1])
  else if (split.length === 2 && split[0] === 'f') fruit.add(split[1])
  input = prompt('Geef een fruit of groente in, of druk op enter om te stoppen:', '')
}

console.log('Groenten:', vegetables)
console.log('Fruit:', fruit)
```

```
// Exercise: finding common elements – array .filter() approach vs Set .intersection()
const array1 = [1, 2, 3, 4, 5];
const array2 = [4, 5, 6, 7, 8];

function findCommonElements(array1, array2) {
  return array1.filter((element) => array2.includes(element));
}

function getIntersection(array1, array2) {
  return Array.from(new Set(array1).intersection(new Set(array2)));
}

console.log('Gemeenschappelijke elementen (array):', findCommonElements(array1, array2));
console.log('Gemeenschappelijke elementen (set):', getIntersection(array1, array2));


// Exercise: Sieve of Eratosthenes – Array(n+1).fill(), map, forEach, splice
const n = Number(prompt('Up until which number do you want to find all prime numbers?'))

const primes = Array(n + 1).fill(true).map((_,i) => (i % 2 !== 0 || i === 2) && i >= 2);

for (let p = 3; p < n; p += 2) {
  if (!primes[p]) continue
  for (let q = Math.pow(p, 2); q <= n; q += 2*p) {
    primes[q] = false;
  }
}

const finalPrimes = []
primes.forEach((isPrime, i) => isPrime && finalPrimes.push(i))
const maxLength = finalPrimes.at(-1).toString().length

const rowSize = 20
while (finalPrimes.length !== 0) {
  const row = finalPrimes.splice(0, rowSize).map(x => x.toString().padEnd(maxLength, ' ')).join
  console.log(row)
}
```

```
// Exercise: 2D array + destructuring — Tic-tac-toe winner detection with ANSI color
const boards = [
  [['X','O','X'],['O','X','O'],['O','X','X']],
  [['X','O','X'],['O','X','O'],['O','X','O']],
  [['X','O','X'],['O','O','O'],['O','X','X']],
  [['X','O','X'],['O','O','X'],['X','X','O']],
]

const winningCombinations = [
  [[0,0],[0,1],[0,2]], [[1,0],[1,1],[1,2]], [[2,0],[2,1],[2,2]], // rows
  [[0,0],[1,0],[2,0]], [[0,1],[1,1],[2,1]], [[0,2],[1,2],[2,2]], // columns
  [[0,0],[1,1],[2,2]], [[0,2],[1,1],[2,0]],                      // diagonals
]

function getTicTacToeWinner(board) {
  for (const combination of winningCombinations) {
    const [a, b, c] = combination
    if (board[a[0]][a[1]] !== board[b[0]][b[1]] || board[a[0]][a[1]] !== board[c[0]][c[1]]) cont
    const winner = board[a[0]][a[1]]
    // ANSI: \x1b[1;32m = bold green, \x1b[0m = reset
    board[a[0]][a[1]] = `\x1b[1;32m${winner}\x1b[0m`
    board[b[0]][b[1]] = `\x1b[1;32m${winner}\x1b[0m`
    board[c[0]][c[1]] = `\x1b[1;32m${winner}\x1b[0m`
    return [winner, board]
  }
  return [undefined, board]
}

for (const board of boards) {
  const [winner, validatedBoard] = getTicTacToeWinner(board)
  console.log(winner ? `De ${winner} speler heeft gewonnen:` : 'Er is geen winnaar:')
  console.table(validatedBoard)
}
```

```
// Exercise: memoized Fibonacci using BigInt — avoids floating-point errors for large n
// BigInt: suffix n (1n) or BigInt(value). Required because JS Number loses precision above MAX_
function fibonacci(n, cache = Array(n + 1).fill(null)) {
  if (cache[n] !== null) return cache[n]
  if (n === 0) return BigInt(0)
  if (n === 1) return 1n

  const fib = fibonacci(n - 1, cache) + fibonacci(n - 2, cache)
  cache[n] = fib
  return fib
}

const n = Number(prompt('Welk Fibonacci-getal wil je berekenen?'))
console.log(fibonacci(n).toLocaleString("nl-BE"))


// Exercise: iterative Fibonacci with BigInt — more efficient for large n
function fibonacci(n) {
  if (n === 0) return 0
  if (n === 1) return 1

  let nMinus2 = 0n
  let nMinus1 = 1n

  for (let i = 2; i <= n; i++) {
    const temp = nMinus1 + nMinus2
    nMinus2 = nMinus1
    nMinus1 = temp
  }
  return nMinus1
}

const n = Number(prompt('Welk Fibonacci-getal wil je berekenen?'))
console.log(fibonacci(n).toLocaleString("nl-BE"))
```

Lecture 3 — Objects

In JavaScript, almost everything is an object. Objects are defined with **object literals**:

```
const person = { firstName: 'Alan', birthYear: 1912 };
```

Access properties with dot notation (`person.firstName`) or bracket notation (`person['firstName']`). You can add or change properties at any time.

`this` works differently than in C#. Inside a regular function used as a callback, `this` no longer refers to the object — it refers to the global context. The fix: always use **arrow functions for callbacks**, since they inherit `this` from the enclosing scope.

```
// ❌ Breaks — `this` is the global context inside the callback
setTimeout(function() { console.log(this.name); }, 1000);

// ✅ Works — arrow function inherits `this` from the object
setTimeout(() => console.log(this.name), 1000);
```

```
// Exercise: object literal with optional property + method using this
const theHobbit = {
  title: 'The Hobbit',
  author: 'J.R.R. Tolkien',
  published: 1937,
  getInfo() {
    const baseInfo = `${this.title} by ${this.author}, published in ${this.published}`;
    return this.wordCount ? `${baseInfo} (${this.wordCount} words)` : baseInfo;
  },
}

const nineteenEightyFour = {
  title: '1984',
  author: 'George Orwell',
  published: 1949,
  wordCount: 88900,
  getInfo() {
    const baseInfo = `${this.title} by ${this.author}, published in ${this.published}`;
    return this.wordCount ? `${baseInfo} (${this.wordCount} words)` : baseInfo;
  },
}

console.log(theHobbit.getInfo());
console.log(nineteenEightyFour.getInfo());

theHobbit.wordCount = 95356;   // adding a property after creation is valid
console.log(theHobbit.getInfo());
```

Prototype chain — JS uses prototype-based inheritance. Every object has a prototype it inherits methods from. Monkey-patching (e.g. `Array.prototype.keepEven = ...`) is possible but discouraged.

```
// Exercise: factory function — returns object with shorthand properties + multiple methods
function createBook(title, author, published, wordCount) {
  return {
    title,           // shorthand: same as title: title
    author,
    published,
    wordCount,
    getInfo() {
      const baseInfo = `${this.title} by ${this.author}, published in ${this.published}`;
      return this.wordCount ? `${baseInfo} (${this.wordCount} words)` : baseInfo;
    },
    bookType() {
      if (!this.wordCount) return 'Unknown'
      if (this.wordCount < 7500)   return 'Short Story'
      if (this.wordCount < 20000)  return 'Novelette'
      if (this.wordCount < 40000)  return 'Novella'
      if (this.wordCount < 250000) return 'Novel'
      return 'Doorstopper'
    }
  };
}

const books = [
  createBook('The Hobbit', 'J.R.R. Tolkien', 1937),
  createBook('1984', 'George Orwell', 1949, 88900),
  createBook('Pride and Prejudice', 'Jane Austen', 1813, 124713),
  createBook('War and Peace', 'Leo Tolstoy', 1867, 544406),
  createBook('The Tell-Tale Heart', 'Edgar Allan Poe', 1843, 2093),
  createBook('The Metamorphosis', 'Franz Kafka', 1915, 22185),
  createBook('Strange Case of Dr Jekyll and Mr Hyde', 'Robert Louis Stevenson', 1886, 13500),
];

books.forEach(book => console.log(`${book.title} is a ${book.bookType()}`));


// Exercise: filter on an array of factory objects using an arrow function
const filterOnYear = (books, maxYear) => books.filter(book => book.published <= maxYear);

filterOnYear(books, 1900).forEach(book => console.log(book.getInfo()));
```

```
// Exercise: monkey-patching Array.prototype — adds .sortNumerical() and .sortObject()
// ⚠️ Discouraged in production; here for educational purposes only
Array.prototype.sortNumerical = function() {
  return this.sort((a, b) => a - b);
}

Array.prototype.sortObject = function(key, type = 'string') {
  if (type === 'number') return this.sort((a, b) => a[key] - b[key]);
  return this.sort((a, b) => a[key].localeCompare(b[key]));
}

console.log(numbers.sortNumerical());
console.log(books.sortObject('title').map(x => x.getInfo()).join('\n'));
console.log(books.sortObject('wordCount', 'number').map(x => x.getInfo()).join('\n'));
```

Iterating over objects: use `Object.keys()`, `Object.values()`, or `Object.entries()` rather than `for...in` (which also traverses the prototype chain).

```
// Exercise: merging two objects — handling key conflicts with Set intersection/difference
function merge(object1, object2) {
  const keys1 = new Set(Object.keys(object1));
  const keys2 = new Set(Object.keys(object2));
  const intersection = keys1.intersection(keys2);
  const result = {}

  for (const key of keys1.difference(intersection)) result[key] = object1[key];
  for (const key of keys2.difference(intersection)) result[key] = object2[key];

  for (const key of intersection) {
    // Conflict: ask user which value to keep
    const choice = prompt(`Key ${key} exists in both objects.\n1: ${object1[key]}\n2: ${object2
    result[key] = choice === '1' ? object1[key] : object2[key];
  }
  return result;
}

const object1 = { a: 1, b: 2, c: 3 };
const object2 = { b: 4, c: 5, d: 6 };
console.log(merge(object1, object2));
```

```
// Exercise: using a Date object + Intl.DateTimeFormat for locale-aware formatting
const date = new Date(prompt("Geef je geboortedatum in (YYYY-MM-DD):"));

const paddedDate  = date.getDate().toString().padStart(2, '0');
const paddedMonth = (date.getMonth() + 1).toString().padStart(2, '0');   // ⚠️ getMonth() is 0-
console.log(`Je geboortedatum is: ${paddedDate}/${paddedMonth}/${date.getFullYear()}`);

const dutchFormatter = new Intl.DateTimeFormat('nl-BE', {
  weekday: 'long', day: 'numeric', month: 'short', year: 'numeric'
})
console.log(`Vandaag is het ${dutchFormatter.format(new Date())}`);

const age = Math.floor((Date.now() - date.getTime()) / (1000 * 60 * 60 * 24));
console.log(`Je bent ${age} dagen oud.`);


// Exercise: character frequency map — object as a counter (dynamic property names)
function getCharacterFrequency(input) {
  const output = {};
  for (let i = 0; i < input.length; i++) {
    const character = input[i];
    if (character === ' ') continue;
    output[character] = output[character] ? output[character] + 1 : 1;
  }
  return output;
}

const input = prompt("Geef een tekst in:");
console.log(getCharacterFrequency(input))
```

```
// Exercise: deep path traversal through nested objects/arrays using a key-path array
function getValueByPath(pathArray, object) {
  let output = object;
  for (const key of pathArray) {
    output = output[key];
  }
  return output;
}

const data = {
  books: [
    { title: 'The Hobbit', genres: ['Fantasy', 'Adventure'] },
    { title: '1984',       genres: ['Dystopian', 'Political Fiction'] },
  ],
  authors: ['J.R.R. Tolkien', 'George Orwell']
}

console.log(getValueByPath(['books', 0, 'title'], data));      // The Hobbit
console.log(getValueByPath(['authors', 2], data));             // undefined (out of range)
console.log(getValueByPath(['books', 1, 'genres', 0], data));  // Dystopian
```

```
// Exercise: interactive object explorer — navigate nested objects via prompt, '..' to go up
function exploreObject(object) {
  const activePath = [];

  while (true) {
    const currentObject = getValueByPath(activePath, object);
    let key = '..'

    if (typeof currentObject !== 'object' && !Array.isArray(currentObject)) {
      console.log({ value: currentObject });
      key = prompt('Reached a leaf node. Press enter to go back or type "exit" to quit', '..');
    } else {
      const availableKeys = Object.keys(currentObject);
      key = prompt(`Available keys: ${availableKeys.join(', ')} — or type 'exit' to quit:`);
    }

    if (key === '..') { activePath.pop(); continue }
    if (key === 'exit') break;

    const availableKeys = Object.keys(currentObject);
    if (!availableKeys.includes(key)) { console.log('Invalid key, retrying...'); continue }

    activePath.push(key);
  }
}

exploreObject(data);
```

Destructuring is a shorthand for extracting values:

```
const { firstName, birthYear } = person;  // order doesn't matter, names must match
const [first, , third] = myArray;         // order matters, names don't
```

Lecture 4 — DOM Manipulation & Events

This is where JS moves into the browser. The **DOM (Document Object Model)** is a tree of HTML elements that JavaScript can read and modify.

Connecting JS to HTML: always place `<script src="...">` at the **bottom** of `<body>` so the HTML loads first.

Finding elements:

```
document.getElementById('myId')
document.querySelector('#myId')              // CSS selector — preferred
document.querySelectorAll('.my-class')       // all matches → NodeList
```

> `getElementsByClassName` / `getElementsByTagName` return live `HTMLCollection` — convert with `Array.from()` to use array methods.

Creating & inserting elements:

```
const el = document.createElement('li');
el.innerText = 'Hello';
el.className = 'some-class';
el.classList.add('active');
parent.appendChild(el);
el.remove();
```

Use `innerText` over `innerHTML` when setting text — `innerHTML` is an XSS risk.

Events: always use `addEventListener` over inline `onclick=` :

```
button.addEventListener('click', (event) => {
  event.preventDefault();
  console.log(event.target);
});
```

Event delegation: one listener on a parent, use `event.target` and `data-*` attributes to identify the clicked child.

```
// Exercise: innerHTML to render arrays into the DOM — fruit and vegetables list
const fruit = ['Appel', 'Kiwi', 'Peer'];
const groenten = ['Wortel', 'Sla', 'Bloemkool'];

document.getElementById('fruit').innerHTML    = "<strong>Fruit</strong>: " + fruit.join(', ');
document.getElementById('groenten').innerHTML = "<strong>Groenten</strong>: " + groenten.join(',
document.getElementById('resultaat').innerHTML = "<strong>Groenten en fruit</strong>: "
+ groenten.concat(fruit).join(', ');


// Exercise: random image switcher — addEventListener, Math.random, while to avoid repeat
const foto = document.getElementById('foto')
const result = document.getElementById('result')
let previousImage = 1

function chooseRandomImage() {
  let newImage = getRandomInteger(1, 3)
  while (newImage === previousImage) {
    newImage = getRandomInteger(1, 3)    // keep rolling until we get a different image
  }
  foto.src = `img/${newImage}.webp`
  previousImage = newImage
  result.innerHTML += `<p>${newImage}</p>`
  document.getElementById('last-photo').innerHTML = newImage
}

function getRandomInteger(min, max) {
  return Math.floor(Math.random() * (max - min + 1)) + min
}

document.getElementById('random-button').addEventListener('click', chooseRandomImage)
```

```
// Exercise: querySelectorAll + forEach to batch-style multiple elements
document.querySelector('.btn.btn-primary').addEventListener('click', changeTitles);
document.querySelector('.btn.btn-warning').addEventListener('click', changeSubtitles);
document.querySelector('.btn.btn-success').addEventListener('click', function () {
  changeTitles();
  changeSubtitles();
});

function changeTitles() {
  document.querySelectorAll('.title').forEach(title => {
    title.style.color = 'red'
    title.style.textDecoration = 'underline'
  });
}

function changeSubtitles() {
  document.querySelectorAll('.subtitle').forEach(title => {
    title.style.color = 'darkblue'
    title.style.fontFamily = 'Verdana'
  });
}


// Exercise: toggle state with a button — lightbulb on/off using src + textContent swap
let lightOn = false
const lightButton = document.getElementById('light-btn')
const img = document.getElementById('afbeelding')

lightButton.addEventListener('click', function () {
  if (lightOn) {
    img.src = 'img/pic_bulboff.gif'
    lightOn = false
    lightButton.textContent = 'Doe het licht aan'
  } else {
    img.src = 'img/pic_bulbon.gif'
    lightOn = true
    lightButton.textContent = 'Doe het licht uit'
  }
})
```

```
// Exercise: shopping cart – createElement, appendChild, reduce for total, input event listener
const shoppingCart = []

addButton.addEventListener('click', function() {
  const product = productInput.value
  const amount  = Number(amountInput.value)
  const price   = Number(priceInput.value)

  if (product) {
    shoppingCart.push({ product, amount, price })
    renderShoppingCart()
    productInput.value = amountInput.value = priceInput.value = ''
  }
})

discountInput.addEventListener('input', function() {
  const code = discountInput.value
  const [_, _discount] = code.split('OFF')
  const discount = Number(_discount)
  const total = shoppingCart.reduce((acc, {amount, price}) => acc + amount * price, 0)

  if (_discount === '' || discount > 60 || isNaN(discount)) {
    document.getElementById('discount-error').classList.remove('d-none')
    totalDisplay.innerHTML = `€${total.toFixed(2)}`
  } else {
    document.getElementById('discount-error').classList.add('d-none')
    discountIndicator.textContent = ` (met ${discount}% korting)`
    totalDisplay.innerHTML = `€${(total * (1 - discount / 100)).toFixed(2)}`
  }
})

function renderShoppingCart() {
  cartDisplay.innerHTML = ''
  shoppingCart.forEach(({product, amount, price}) => {
    cartDisplay.appendChild(createListItem(`${product} (${amount} x €${price.toFixed(2)})
    = €${(amount * price).toFixed(2)}`))
  })
  const total = shoppingCart.reduce((acc, {amount, price}) => acc + amount * price, 0)
  totalDisplay.textContent = `€${total.toFixed(2)}`
}

function createListItem(text) {
  const listItem = document.createElement('li')
```

```
  listItem.textContent = text
  return listItem
}


// Exercise: Array.from + querySelectorAll — dice roll game with classList manipulation
const players = document.querySelectorAll('.card');
const result  = document.getElementById('result');

document.getElementById('play').addEventListener('click', function () {
  reset()
  const guess = Number(prompt('Hoeveel spelers denk je dat er een 6 gaan gooien? (1-12)'))
  const rolls = Array.from({length: 12}, () => getRandomInteger(1, 6));
  let correct = 0

  for (let i = 0; i < 12; i++) {
    if (rolls[i] !== 6) continue
    players[i].classList.remove('bg-secondary')
    players[i].classList.add('bg-success')
    correct++
  }

  result.textContent = `${correct} speler(s) gooide een 6`
  alert(correct === guess ? 'Proficiat! Je hebt het juist geraden!' : 'Helaas, je hebt het fout
});

document.getElementById('reset').addEventListener('click', reset);

function reset() {
  players.forEach(player => {
    player.classList.remove('bg-success')
    player.classList.add('bg-secondary')
    result.textContent = ''
  });
}

function getRandomInteger(min, max) {
  return Math.floor(Math.random() * (max - min + 1)) + min
}
```

Teacher examples — DOM + data arrays + card rendering pattern:

The following examples from the teacher demonstrate the core exam pattern: store data in an array of objects, render Bootstrap cards via `createElement` / `appendChild`, and hook up search/add/remove interactions.

```
// Teacher example: render an array of student objects as Bootstrap cards
const students = [
  { naam: 'Alice', leeftijd: 19, afstudeerrichting: 'Wiskunde' },
  { naam: 'Bart',  leeftijd: 23, afstudeerrichting: 'Boekhouden' },
  { naam: 'Hannelore', leeftijd: 18, afstudeerrichting: 'IT' },
]
const studentscontainer = document.querySelector('#students');

function showStudents() {
  students.forEach(stud => {
    const card = document.createElement('div')
    card.className = 'card m-2'
    card.style.width = '18rem'

    const cardBody = document.createElement('div')
    cardBody.className = 'card-body'

    const cardTitle = document.createElement('h5')
    cardTitle.className = 'card-title'
    cardTitle.textContent = stud.naam

    const cardText = document.createElement('p')
    cardText.className = 'card-text'
    cardText.textContent = `Leeftijd: ${stud.leeftijd}, Afstudeerrichting: ${stud.afstudeerricht

    cardBody.appendChild(cardTitle)
    cardBody.appendChild(cardText)
    card.appendChild(cardBody)
    studentscontainer.appendChild(card)
  });
}

showStudents();
```

```
// Teacher example: search with 'input' event — filters on every keystroke
// Note: 'input' fires on each keystroke; 'change' only fires when leaving the field
const searchBar = document.getElementById('search-bar')
searchBar.addEventListener('input', searchStudents)

function searchStudents() {
  const query = searchBar.value.toLowerCase()
  const filteredStudents = students.filter(student =>
    student.naam.toLowerCase().includes(query) ||
    student.afstudeerrichting.toLowerCase().includes(query)
  )
  studentsContainer.innerHTML = ""
  filteredStudents.forEach(stud => studentsContainer.appendChild(createCard(stud)))
}


// Teacher example: add student via form, push to array, re-render — extract into createCard()
// ⚠️ Bad pattern: re-declaring 'let naam' inside the event listener shadows the outer variable
// Good pattern from modal.js: use different variable names inside the handler
const formAdd = document.querySelector('.student-toevoegen')
const naamInput = document.querySelector('#naam')
const leeftijdInput = document.querySelector('#leeftijd')
const afstudeerrichtingInput = document.querySelector('#afstudeerrichting')

formAdd.addEventListener('click', function () {
  const naam = naamInput.value
  const leeftijd = leeftijdInput.value
  const afstudeerrichting = afstudeerrichtingInput.value
  addStudent(naam, leeftijd, afstudeerrichting)
})

function addStudent(naam, leeftijd, afstudeerrichting) {
  students.push({ naam, leeftijd, afstudeerrichting })
  studentsContainer.textContent = ""   // clear before re-rendering
  showStudents()
}
```

```
// Teacher example (Les Niels2): extract createCard() as a reusable function
// Separating concerns makes both showStudents() and searchStudents() simpler
function createCard(student) {
  const card = document.createElement('div')
  card.className = 'card m-2'
  card.style.width = '18rem'

  const cardBody = document.createElement('div')
  cardBody.className = 'card-body'

  const cardTitle = document.createElement('h5')
  cardTitle.className = 'card-title'
  cardTitle.textContent = student.naam

  const cardText = document.createElement('p')
  cardText.className = 'card-text'
  cardText.textContent = `Leeftijd: ${student.leeftijd}, Afstudeerrichting: ${student.afstudeerı

  cardBody.appendChild(cardTitle)
  cardBody.appendChild(cardText)
  card.appendChild(cardBody)
  return card   // ⚠️ always return the element — forgetting this is a common bug
}

function showStudents() {
  students.forEach(stud => studentsContainer.appendChild(createCard(stud)));
}

function searchStudents() {
  const query = searchBar.value.toLowerCase()
  const filteredStudents = students.filter(student =>
    student.naam.toLowerCase().includes(query) ||
    student.afstudeerrichting.toLowerCase().includes(query)
  )
  studentsContainer.innerHTML = ""
  filteredStudents.forEach(stud => studentsContainer.appendChild(createCard(stud)))
}
```

```
// Teacher example (Les Niels2): Bootstrap modal — open dialog for data entry, dismiss on submit
// The HTML uses data-bs-toggle="modal" data-bs-target="#exampleModal"
// data-bs-dismiss="modal" on the submit button closes the modal automatically
// Script side: identical to addStudent pattern above — just wired to a modal form
```

JavaScript is **single-threaded** but handles async work via the event loop. Long operations must be non-blocking.

Promises represent a future value — like C#'s `Task<T>`. Three states: Pending, Fulfilled, Rejected.

```
fetch('https://api.example.com/data')
  .then(response => response.json())
  .then(data => console.log(data))
  .catch(error => console.error(error))
  .finally(() => console.log('done'));
```

`async` / `await` — equivalent to C#'s `async` / `await`:

```
async function loadData() {
  try {
    const response = await fetch('https://api.example.com/data');
    const data = await response.json();
    console.log(data);
  } catch (error) {
    console.error(error);
  }
}
```

`Promise.all()` waits for multiple promises to all resolve — equivalent to `Task.WhenAll()`.

Fetch for POST:

```
fetch(url, {
  method: 'POST',
  body: JSON.stringify({ title: 'test' }),
  headers: { 'Content-Type': 'application/json' }
});
```

```
// Exercise: basic fetch + async/await — render a list from a public API
// mouseover/mouseout used here for inline hover styling (in production, use CSS classes)
async function getUsers() {
  const response = await fetch('https://jsonplaceholder.typicode.com/users');
  const users = await response.json();
  const userList = document.getElementById('user-list');

  users.forEach(user => {
    const listItem = document.createElement('li');
    listItem.textContent = user.name;
    userList.appendChild(listItem);

    listItem.addEventListener('mouseover', () => listItem.style.backgroundColor = 'yellow')
    listItem.addEventListener('mouseout',  () => listItem.style.backgroundColor = 'unset')
  });
}
getUsers()


// Exercise: fetch + try/catch — show a Bootstrap error alert when the request fails
async function getStarWarsCharacters() {
  try {
    const response = await fetch('https://swapi.dev/api/people/');
    const data = await response.json();
    data.results.forEach(character => addCharacterToList(character));
  } catch (e) {
    const errorMessage = document.getElementById('error-message');
    errorMessage.classList.remove('d-none');
    errorMessage.textContent = 'Er is een fout opgetreden bij het ophalen van de data.';
  }
}

function addCharacterToList(character) {
  const listItem = document.createElement('li');
  listItem.classList.add('list-group-item');
  listItem.textContent = `${character.name}${character.gender === 'n/a' ? '' : ' (' + character
  list.appendChild(listItem);
}
getStarWarsCharacters()
```

```
// Exercise: chained fetches + loading spinner — await a deliberate delay, then fetch + filter
async function getImageURIs() {
  await new Promise(r => setTimeout(r, 2000))   // artificial delay to keep spinner visible

  const response = await fetch('https://picsum.photos/v2/list?limit=25')
  const images = await response.json()
  return images.filter(image => image.height < image.width)  // landscape only
}

async function displayImages() {
  const images = await getImageURIs()
  spinner.hidden = true

  for (const { author, download_url, height, width, url } of images) {
    results.innerHTML += `
      <div class="col-md-4">
        <div class="card mb-4 shadow-sm">
          <img src="${download_url}" class="img-responsive">
          <div class="card-body">
            <p class="card-text">${author}</p>
            <div class="d-flex justify-content-between align-items-center">
              <a href="${url}" target="_blank" class="btn btn-primary" role="button">Meer info</
              <small class="text-muted">${height} x ${width}</small>
            </div>
          </div>
        </div>
      </div>`
  }
  results.hidden = false
}

displayImages()
```

Teacher examples (Les Niels2) — fetch from an API + search + remove pattern:

```
// Teacher example: fetch from external API, store globally, then render + live search
let students = [];
const list = document.getElementById("student-list");
const searchInput = document.getElementById("search");

async function fetchStudents() {
  const result = await fetch('https://tm-statweb.be/students.php');
  students = await result.json();
}

function createStudentItem(student) {
  const item = document.createElement("li");
  item.classList.add("list-group-item", "d-flex", "justify-content-between", "align-items-center

  const name = document.createElement("span");
  name.textContent = student.name + " / " + student.level;
  item.appendChild(name);

  const removeButton = document.createElement("button");
  removeButton.classList.add("btn", "btn-danger", "btn-sm");
  removeButton.textContent = "Remove";
  item.appendChild(removeButton);

  removeButton.addEventListener("click", () => {
    list.removeChild(item);   // removes only the DOM node — the data array is unchanged
  })
  return item;
}

function renderStudents() {
  const searchValue = searchInput.value.toLowerCase();
  const filteredStudents = students.filter(stud =>
    stud.name.toLowerCase().includes(searchValue) ||
    stud.level.toLowerCase().includes(searchValue)
  );

  list.innerHTML = "";
  filteredStudents.forEach(student => list.appendChild(createStudentItem(student)));
}

// Search on every keystroke; clear search box with a double-click
searchInput.addEventListener("input",    () => renderStudents());
searchInput.addEventListener("dblclick", () => { searchInput.value = ""; renderStudents(); })
```

```
// .then() chains the render after the fetch completes — equivalent to await inside async
fetchStudents().then(() => renderStudents());


// Teacher example (cards.js): same pattern but rendering Bootstrap image cards with avatar
function createStudentItem(student) {
  const item = document.createElement("div");
  item.classList.add("col");

  const divCard = document.createElement("div");
  divCard.classList.add("card", "mb-3");
  item.appendChild(divCard);

  const cardImg = document.createElement("img");
  cardImg.src = student.avatar;
  cardImg.classList.add("card-img-top");
  divCard.appendChild(cardImg);

  const cardBody = document.createElement("div");
  cardBody.classList.add("card-body");
  divCard.appendChild(cardBody);

  const cardH5 = document.createElement("h5");
  cardH5.classList.add("card-title");
  cardH5.textContent = student.name;
  cardBody.appendChild(cardH5);

  const cardP = document.createElement("p");
  cardP.classList.add("card-text");
  cardP.textContent = `Level: ${student.level}`;
  cardBody.appendChild(cardP);

  const removeButton = document.createElement("button");
  removeButton.classList.add("btn", "btn-danger", "btn-sm");
  removeButton.textContent = "Remove";
  cardBody.appendChild(removeButton);

  removeButton.addEventListener("click", () => item.remove())  // .remove() — no parent needed

  return item;
}
```

Oefenexamen (April 2025) — Webshop: fetch + render + filter + cart:

This is the full exam exercise. It combines Lecture 4 (DOM, events) and Lecture 5 (fetch, async/await, try/catch). The pattern is identical to the teacher examples but extended with category filtering and a shopping cart.

```
// Exam: fetch products from local JSON, show error alert on failure
async function getProducts() {
  try {
    const response = await fetch('./data/products.json');
    const data = await response.json();
    return data.products;
  } catch (error) {
    const errorMessage = document.getElementById('errorMessage')
    errorMessage.classList.remove('d-none')
    errorMessage.textContent = 'Error while loading products.'
  }
}

// Exam: render product cards — make filter buttons visible only after successful load
function renderProducts(datarender) {
  document.getElementById('filter-buttons').classList.remove('d-none');
  const grid = document.getElementById("product-list");
  grid.innerHTML = '';

  datarender.forEach(product => {
    const card = document.createElement("div")
    card.className = "card m-2"
    card.style.width = "18rem"

    const cardImg = document.createElement("img")
    cardImg.className = "card-img-top"
    cardImg.alt = product.name
    cardImg.src = product.image

    const cardBody = document.createElement("div")
    cardBody.className = "card-body"

    const cardTitle = document.createElement("h5")
    cardTitle.className = "card-title"
    cardTitle.textContent = product.name

    const cardText = document.createElement("p")
    cardText.className = "card-text"
    cardText.textContent = product.description

    // data-* attributes used for identification without polluting JS state
    const cardButton = document.createElement("button")
    cardButton.className = "btn btn-dark btn-sm mb-3"
```

```
    cardButton.textContent = 'Add to cart'
    cardButton.setAttribute('data-id', product.id)
    cardButton.addEventListener('click', () => addToCart(product.id));

    card.appendChild(cardImg);
    card.appendChild(cardBody);
    card.appendChild(cardTitle);
    card.appendChild(cardText);
    card.appendChild(cardButton);

    grid.appendChild(card)
  });
}

// Exam: init — fetch, render all, then wire up category filter buttons
// button.dataset.category reads the data-category HTML attribute
async function init() {
  const products = await getProducts();
  if (!products || products.length === 0) return;

  renderProducts(products);

  document.querySelectorAll('.filter-btn').forEach(button => {
    button.addEventListener('click', () => {
      const category = button.dataset.category;
      if (category === 'all') {
        renderProducts(products)
      } else {
        renderProducts(products.filter(p => p.category === category));
      }
    });
  });
}

init();
```

Lecture 6 — TypeScript

TypeScript is a **superset of JavaScript** with static typing. Think of it as JavaScript + C#'s type system. It compiles to plain JS — no browser runs TypeScript directly.

```ts
let count: number = 0;
let flags: boolean[] = [true, false];
let coords: [number, number] = [10, 20];   // tuple

interface Book {
  title: string;
  year: number;
  series?: string;    // optional property
}

type Position = 'left' | 'right' | 'top' | 'bottom';   // union of literals

function greet(name: string): string { return `Hello ${name}`; }
async function deleteUser(id: string): Promise<void> { ... }
```

TypeScript infers types where possible. The `any` type disables checking — use sparingly. `unknown` is safer. Optional chaining (`?.`) and nullish coalescing (`??`) are enforced more strictly by the compiler.

Lecture 7 — Vite

Vite is a build tool and dev server for TypeScript projects. Bootstrap with `npm create vite` , choose **Vanilla + TypeScript**.

File/Folder	Purpose
`index.html`	Single entry point; `<div id="app">` is the root
`src/main.ts`	JS/TS entry point via `<script type="module">`
`package.json`	Dependencies and scripts (`dev` , `build`)
`node_modules/`	Never commit — restore with `npm install`

ES Modules:

```ts
export const myFunc = () => { ... };       // named export
export default myClass;                    // default export (one per file)
import { myFunc } from './myFile.ts';
import myClass from './myFile.ts';
```

Vite-specific:

```
import homePage from './home.html?raw';     // import HTML as a string
root.innerHTML = homePage;

const input = document.querySelector<HTMLInputElement>('#myInput')!;
// ! = non-null assertion: tells TS "this element definitely exists"
```

Lecture 8 — Multipage Apps

Introduces a clean **architecture** for multi-page SPAs using abstract classes.

`Page` **abstract class**: each page extends it, passing its HTML to `super()`. The `render()` method injects it into `#app`. A `cleanup()` method unregisters observers when navigating away.

`CustomElement` **abstract class**: extends the native `HTMLElement`, enabling reusable components like `<custom-navbar>`. Register with `customElements.define('custom-navbar', Navbar)`. Pass data in via `setAttribute()`, react to changes via `attributeChangedCallback`, emit events outward via `CustomEvent` + `dispatchEvent`.

`Router` **class**: maps paths to page constructors, intercepts `data-link` clicks, and uses `window.history.pushState` to navigate without page reloads.

```
new Router({
  '/': HomePage,
  '/library': LibraryPage,
  '/authors': AuthorsPage,
});
```

Lecture 9 — Data Management

Introduces a generic **repository pattern** and the **Observer pattern** for reactive UI updates.

`PersistenceProvider<T>` — abstract generic CRUD class. `T` must have an `id: string` (via `Persistable`). `Omit<T, 'id'>` strips the ID field for create operations.

Implementation	Storage	Survives page refresh?
`MemoryPersistenceProvider<T>`	In-memory	❌
`LocalStoragePersistenceProvider<T>`	Browser localStorage	✅ (~5MB)

Observer pattern: pages subscribe to data changes and re-render automatically. `addObserver()` returns an unsubscribe function — always store and call it on `cleanup()` to prevent memory leaks and duplicate renders.

```
this.unsubscribe.push(
  bookProvider.addObserver(books => { this.#books = books; this.render(); })
);
this.unsubscribe.forEach(fn => fn());   // call on page cleanup
```

📎 Addendum — Topics Worth Extra Attention

These are recurring sources of confusion highlighted in the course slides. Worth studying carefully before exams or exercises.

A1 — `const` vs `let` vs `var` (scope table)

Keyword	Global scope	Block scope	Can reassign	Can redeclare
`const`	❌	✅	❌	❌
`let`	❌	✅	✅	❌
`var`	✅	❌	✅	✅

`var` leaks out of `if` / `for` / `while` blocks into the surrounding function or global scope. `const` and `let` are block-scoped, like C#. Always use `const` first; switch to `let` only when you need to reassign.

A2 — `typeof null` is `"object"` — a known bug

```
typeof null   // "object" ⚠️ — null is NOT an object, this is a historical bug
```

To safely check for null, always use strict equality:

```
if (value === null) { ... }          // ✅ correct
if (typeof value === 'object') { }   // ⚠️ also matches null!
```

A3 — `prompt()` always returns a string

This is a very common source of bugs:

```
const age = prompt('How old are you?');
console.log(age + 1);    // "251" ❌ — string concatenation, not addition!

const age = Number(prompt('How old are you?'));
console.log(age + 1);    // 26 ✅
```

Always wrap with `Number()` when you expect numeric input from `prompt()`.

A4 — Hoisting

Function **declarations** are hoisted — you can call them before they appear in the file:

```
greet();  // ✅ works

function greet() { console.log('Hello!'); }
```

Arrow functions and function expressions assigned to `const` / `let` are NOT hoisted:

```
greet();  // ❌ ReferenceError

const greet = () => console.log('Hello!');
```

A5 — `sort()` does not sort numbers correctly by default

`.sort()` converts elements to strings and compares them by ASCII/Unicode value:

```
[5, 4, 10000, 20, 1, 32].sort()
// → [1, 10000, 20, 32, 4, 5]  ❌ wrong numerical order!

[5, 4, 10000, 20, 1, 32].sort((a, b) => a - b)
// → [1, 4, 5, 20, 32, 10000]  ✅ correct
```

The comparator `(a, b) => a - b` returns negative if `a` should come first, positive if `b` should come first, `0` if equal.

A6 — Sparse arrays and loop behaviour

A sparse array has empty slots:

```
const sparse = [1, , 3];    // index 1 is empty
```

Different loops handle empty slots differently:

Loop	Result
`for...of`	`1, undefined, 3` — shows the gap
`for...in`	`1, 3` — silently skips the gap
`.forEach()`	`1, 3` — silently skips the gap

This can cause subtle bugs if you accidentally create sparse arrays (e.g. by assigning to a high index).

A7 — `reduce()` initial value matters

```
const nums = [1, 2, 3, 4, 5];

nums.reduce((acc, cur) => acc + cur, 0);    // 15  ✅ — neutral for addition is 0
nums.reduce((acc, cur) => acc * cur, 0);    // 0   ❌ — 0 × anything = 0!
nums.reduce((acc, cur) => acc * cur, 1);    // 120 ✅ — neutral for multiplication is 1
nums.reduce((acc, cur) => acc + cur, '');   // "12345" ✅ — neutral for string join is ''
```

Always think about what the neutral element is for your operation.

A8 — Variable scope: inner vs outer

A variable declared with `const` / `let` inside a function is **local** — it does not affect outer variables of the same name:

```
let username = 'John';

function showMessage() {
  const username = 'Bob';    // local — shadows the outer variable
  console.log(username);     // 'Bob'
}

showMessage();
console.log(username);       // 'John' — outer unchanged
```

But assigning to an outer variable **without** a local declaration modifies it:

```
let username = 'John';

function showMessage() {
  username = 'Bob';          // modifies the OUTER variable!
}

showMessage();
console.log(username);       // 'Bob' — changed!
```

Prefer declaring variables locally with `const` / `let` to avoid unexpected side effects.

Last updated: March 2026 — Course slides: Thomas More (PIT Graduaten) — Website: Sebastiaan Henau